武汉理工大学数统学院数学系课程实验报告

课 程 名 称：数字图像处理

班级	**数学 jd2401**	**日　期**	**2026.11.6**	**成绩评定**	
姓名	**蔡斌**	**实验室**		**老师签名**	
实验名称	实验二 图像增强的频域方法				
所用软件	Python 3.9.7、NumPy 2.0.2、Pillow 11.2.1、Matplotlib 3.9.4；MATLAB R2023b 样例图像				
实验目的及内容	目的：掌握频域平滑、锐化和周期噪声陷波。内容：傅里叶频谱、六种径向滤波器、高频强调及四幅周期噪声图处理，详见附页。				
实验原理步骤、核心代码	实验公式、参数、运行结果、图表分析及 Python 核心源代码详见附页。				

实验结果及分析	滤波结果图、PSNR 数据及分析见附页。

备注：不交此报告者，本次实验为“不合格”。

实验二 图像增强的频域方法（附页）

实验条件与数据

程序运行于 Windows 10、Python 3.9.7、NumPy 2.0.2、Pillow 11.2.1、Matplotlib 3.9.4。低通与高通实验使用 MATLAB R2023b `imdata` 目录内的 256×256 `cameraman.tif`；噪声随机种子为 20261106。任务提供的 7.PNG、10.jpg、11.jpg、16.jpg 保留原始尺寸，用于频谱峰定位和陷波。

频域滤波把图像变换为复数频谱 F(u,v)，逐频率乘以实值传递函数 H(u,v)，再作逆变换：$g(x,y)=F^{-1}\{H(u,v)F(u,v)\}$。径向距离 $D(u,v)=\sqrt{u^2+v^2}$ 以中心化频谱零点为原点，D0 的单位是频率 bin。

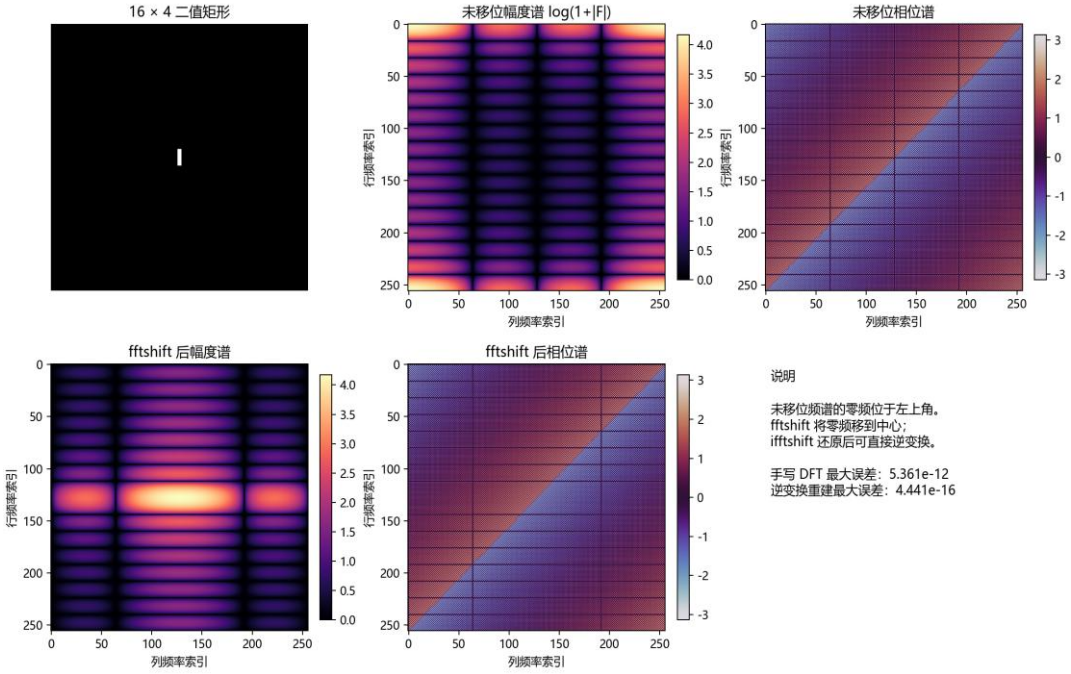
本次比较的低通函数为：理想低通 $H=1(D\le D0)$；巴特沃斯低通 $H=1/[1+(D/D0)^{(2n)}]$，n=2；高斯低通 $H=\exp(-D^2/(2D0^2))$。高通由 HHP=1−HLP 得到；高频强调为 Hhfe=a+b·HHP。

1. 二值图像的傅里叶变换

在 256×256 黑色背景中心放置 16 行×4 列白色矩形。除调用 `numpy.fft.fft2` 外，还按 DFT 定义构造行、列变换矩阵计算二维结果。手写 DFT 与 fft2 的最大绝对误差为 5.361×10^{-12}；经过 fftshift/ifftshift 后再逆变换，像素最大误差为 4.441×10^{-16}。

取 log(1+|F|) 显示幅度谱，使用 angle(F) 显示相位谱。未移位频谱的零频在左上角；fftshift 把它移到中心，便于按半径观察频率分量。

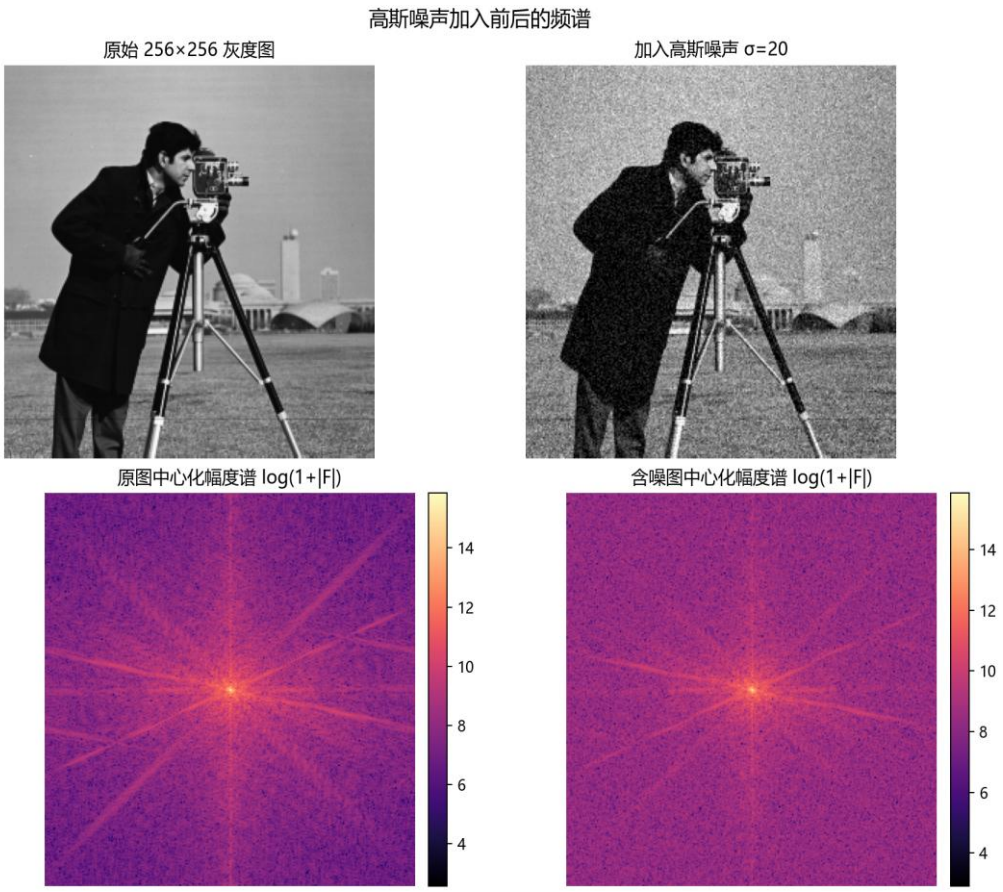
图 1 二值矩形、未移位与中心化幅度谱/相位谱；右下列出手写 DFT 和逆变换误差



2. 高斯噪声与低通滤波

对 cameraman.tif 加入零均值、高斯标准差 σ=20 的独立噪声，并截断到 [0,255]。含噪图相对加噪前原图的 PSNR 为 22.449 dB。频谱图中，噪声加入后背景频率能量上升。

图 2 高斯噪声加入前后的图像与中心化对数幅度谱

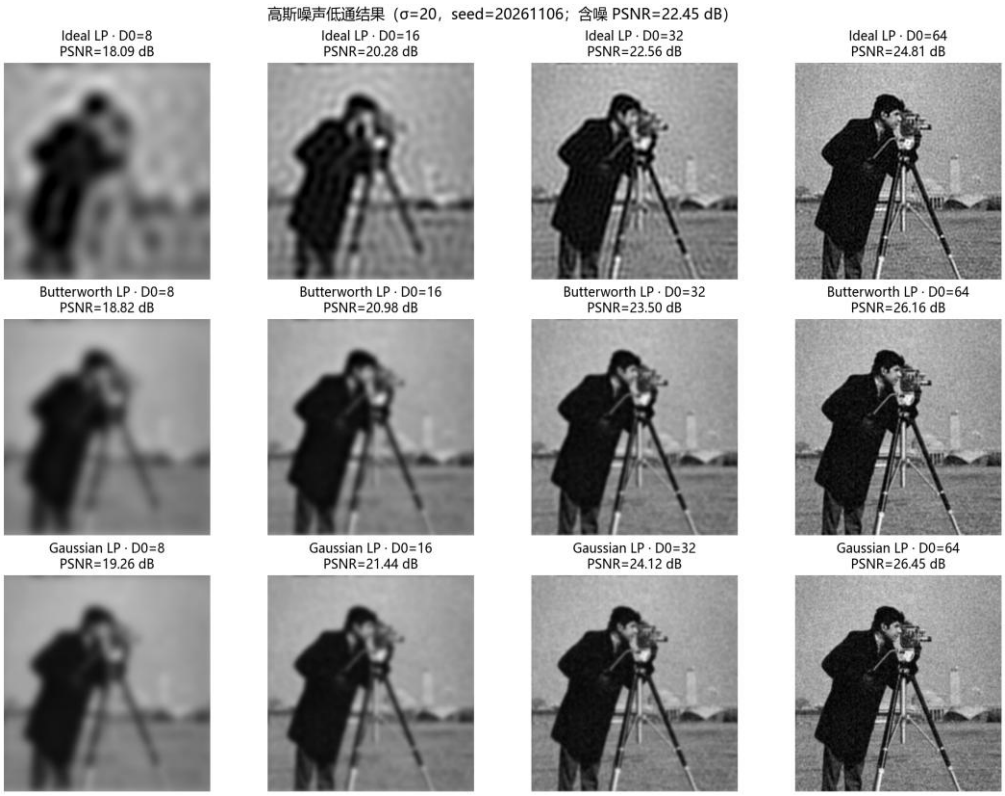


三个低通滤波器使用 D0=8、16、32、64 bin；含噪图滤波后均以原始 cameraman.tif 为 PSNR 参照。表中 PSNR/MSE 按未裁剪浮点输出计算，保存的 PNG 则转换为 8 位灰度图；数值来自同一次固定种子运行。

表 1 三类低通滤波器的 PSNR 对比（dB）

低通类型	D0=8	D0=16	D0=32	D0=64
理想	18.089	20.284	22.563	24.805
巴特沃斯（n=2）	18.815	20.982	23.496	26.162
高斯	19.261	21.443	24.117	26.455

图 3 理想、巴特沃斯和高斯低通在四个截止频率下的输出（各图标题含 PSNR）

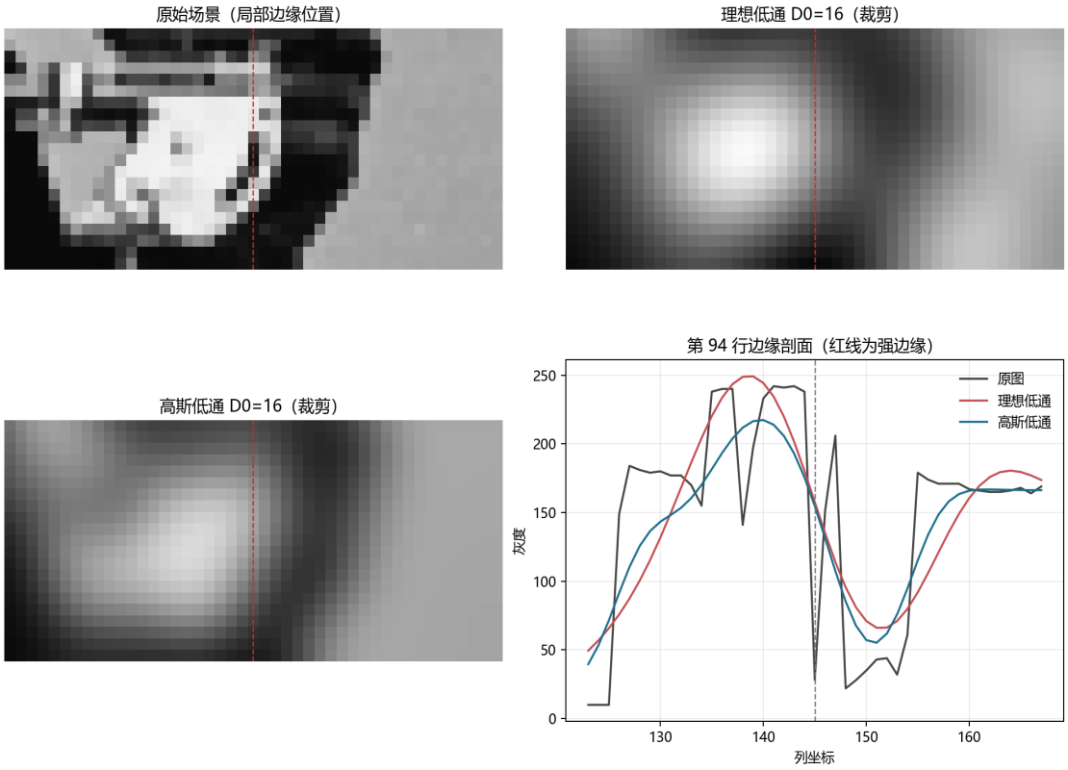


在本次 σ=20 的图像和 D0=8–64 的范围内，三类滤波器的 PSNR 都随 D0 增大而升高；高斯低通 D0=64 得到 26.455 dB，是本组最高值。小 D0 压低更多高频，轮廓变软；增大 D0 后细节恢复，噪声也会有更多残留。因此 D0 的效果需要结合图像和噪声强度判断。

3. 理想低通的振铃现象

理想低通在频域以圆形边界突然从 1 跳到 0；其空间域冲激响应具有延伸的振荡旁瓣。图像边缘与该响应卷积后，会在亮暗跳变附近出现过冲和欠冲。下图在同一条高对比边缘上比较原图、D0=16 理想低通和高斯低通；红色曲线越过原图亮度范围后再回摆，体现了振铃。

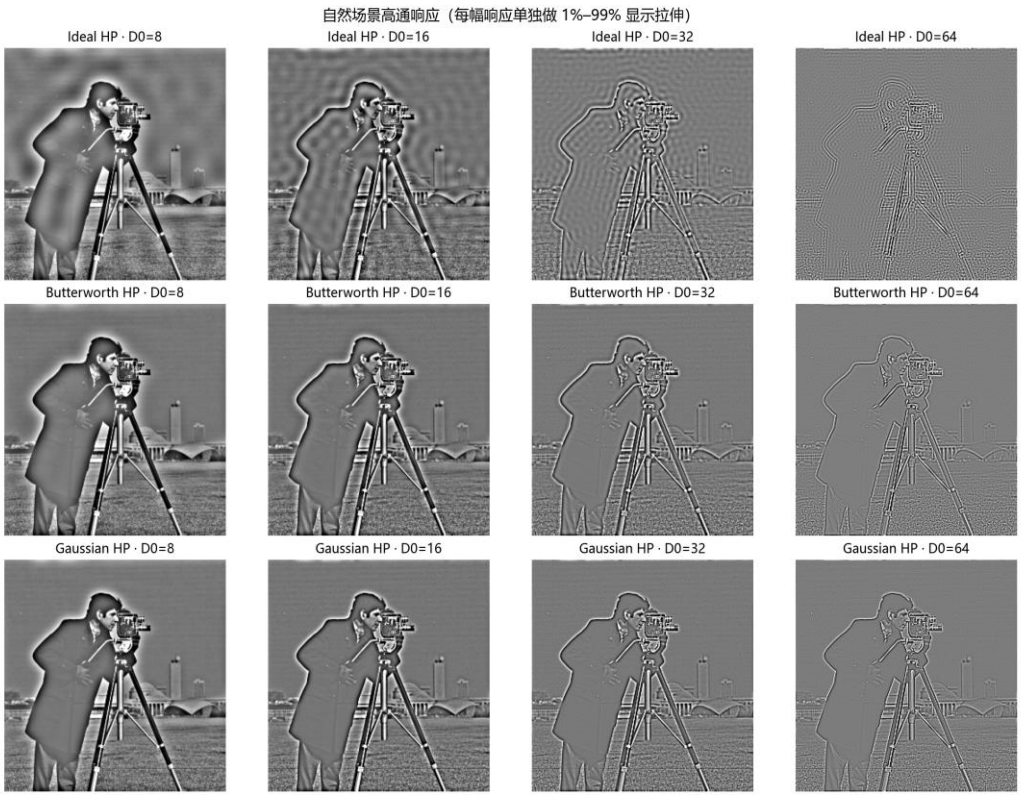
图 4 cameraman.tif 局部边缘和灰度剖面；理想低通产生的振荡比平滑过渡的高斯低通明显



4. 高通滤波与高频强调

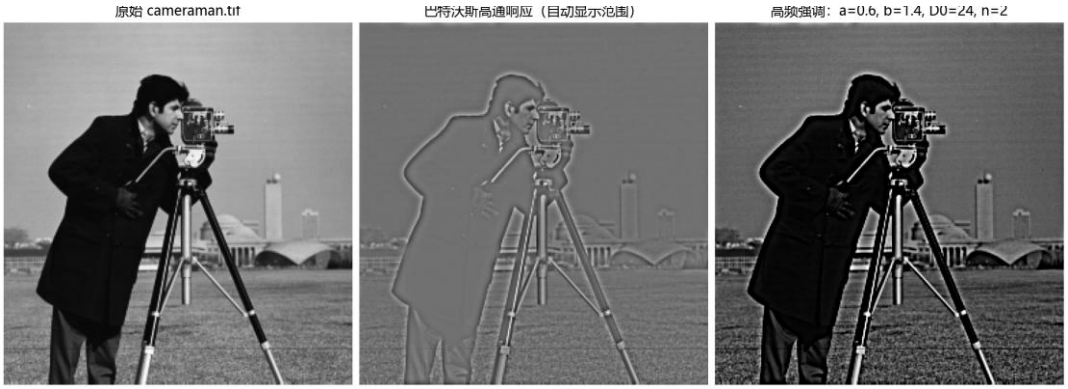
对自然场景 cameraman.tif 分别使用理想、巴特沃斯和高斯高通，D0 仍取 8、16、32、64 bin。高通响应图按每幅响应的 1% – 99% 百分位范围独立线性拉伸，便于观察不同截止频率下的纹理响应；CSV 记录未拉伸响应的最小值和最大值。D0 小时只抑制靠近原点的一小圈频率，边缘和细纹响应较强；D0 增大后抑制的低频范围变宽。

图 5 三类高通滤波器在四个截止频率下的响应（独立显示拉伸）



高频强调使用 Hhfe=0.6+1.4HHP，巴特沃斯阶数 n=2、D0=24。0.6 倍低频基线与 1.4 倍高通分量共同提高边缘对比；由于 Hhfe(0)=0.6，图像均值由 118.72 降至 71.23。滤波结果中约 8.70% 的像素超出 [0,255]，保存为 8 位图像时会被裁剪。

图 6 原场景、高通响应和高频强调结果；增强参数 a=0.6、b=1.4、D0=24、n=2



5. 四幅周期噪声图的陷波处理

实值图像的周期干扰在傅里叶幅度谱中成共轭亮峰。程序将选中的中心 (v,u) 与 (−v,−u) 同时加入高斯陷波器；每个中心的抑制量为 $1-\exp(-Dk^2/(2D0^2))$，成对相乘后得到实值、中心对称的传递函数。界面支持左键选峰、右键删除最近一组、拖动 D0 滑块和保存输出。频谱峰先由程序列出候选，再按图像中明显的孤立峰人工选择。各图保留输入尺寸；坐标是中心化频谱中的行偏移 v 和列偏移 u，单位为 bin。

表 2 四幅输入图像的陷波中心与半径

输入图像	H×W	陷波中心 (v,u)	D0
7.PNG	275×272	(13,22)、(25,-44)	3.0
10.jpg	471×640	(61,-83)、(62,82)、(2,165)、(123,-1)、(1,125)	8.0
11.jpg	164×220	(25,0)、(51,0)、(75,0)	5.0
16.jpg	512×512	(0,16)、(5,5)	3.0

图 7 四幅指定图像的输入、Gaussian notch 输出与陷波位置响应

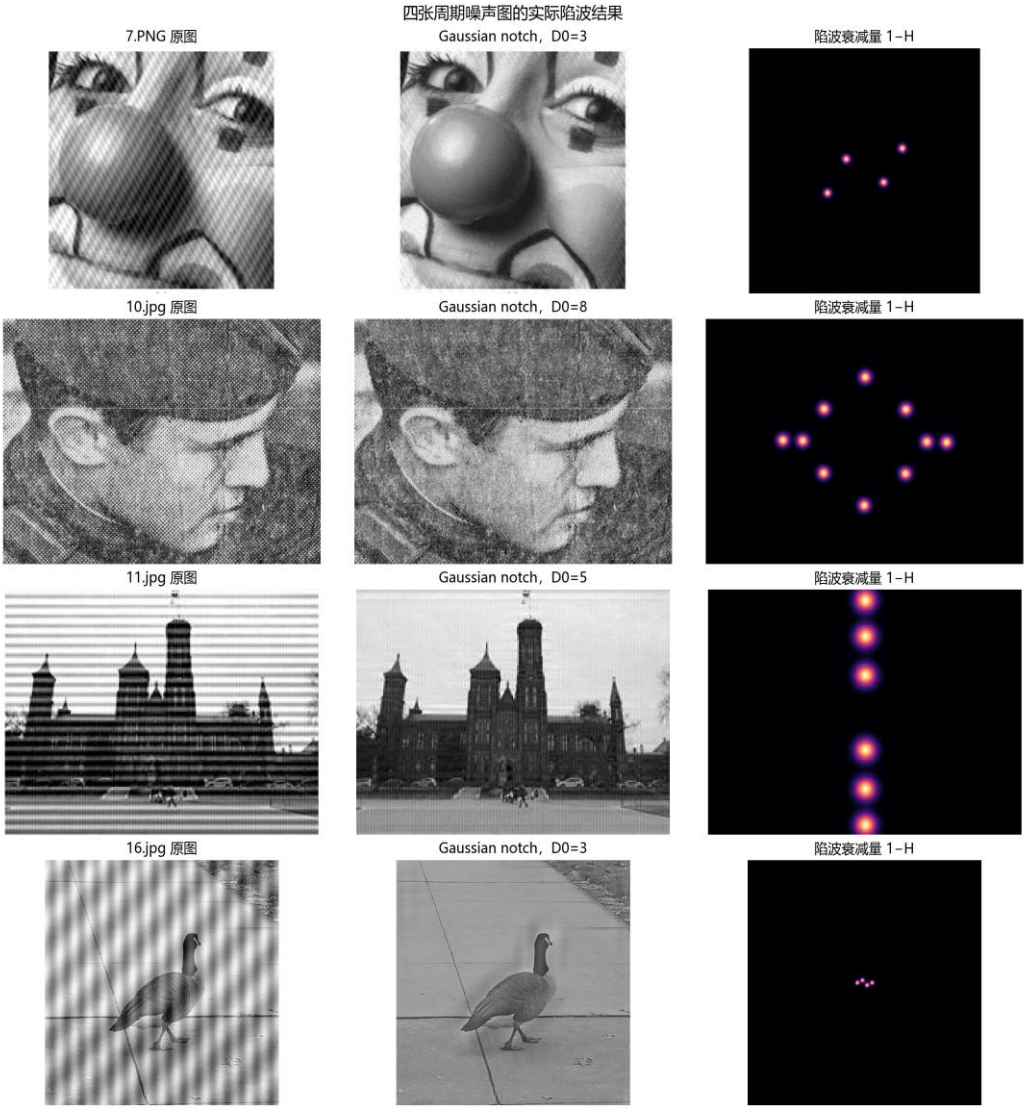
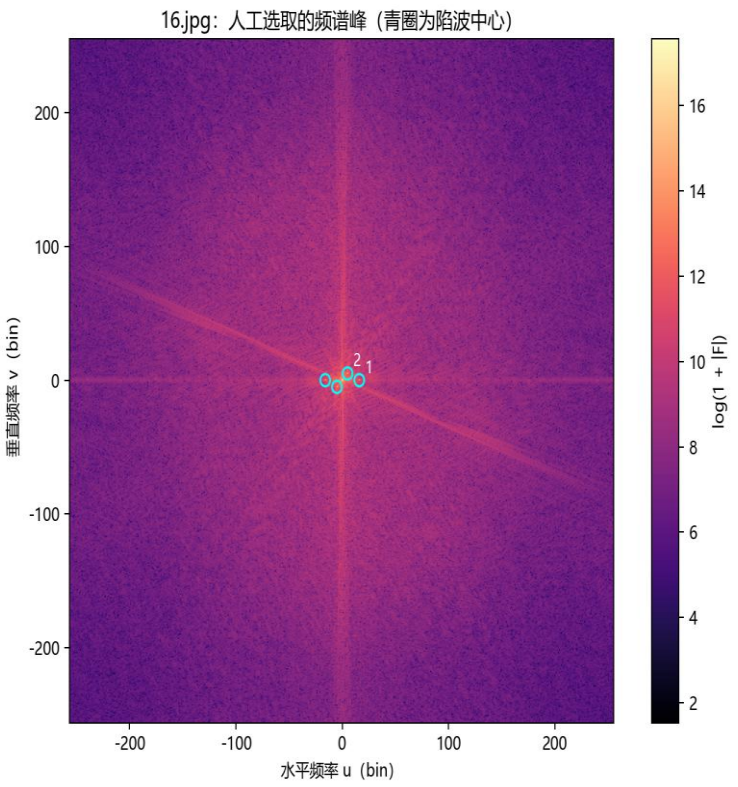


图 8 16.jpg 的中心化对数频谱；青圈标出实际陷波中心及其共轭位置


16.jpg：人工选取的频谱峰（青圈为陷波中心）
垂直频率 v（bin）
水平频率 u（bin）
log(1 + |F|)
200
100
0
-100
-200
16
14
12
10
8
6
4
2
1
2

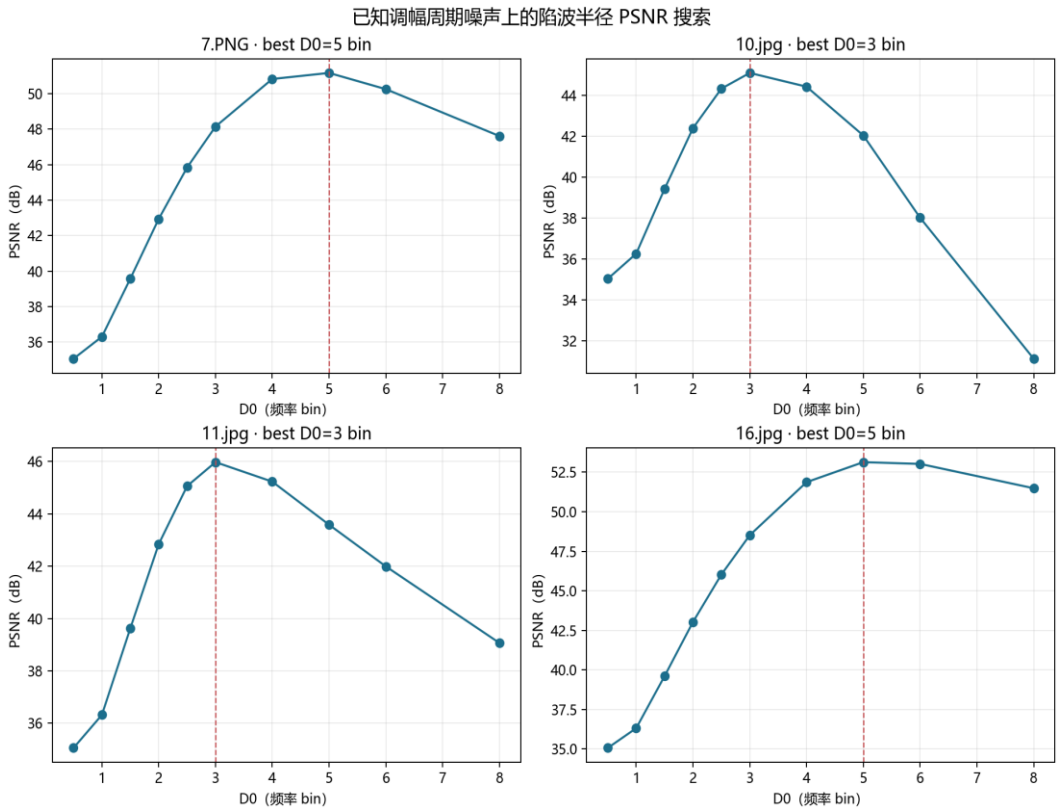
6. 陷波半径的 PSNR 搜索

四幅给定周期噪声图均没有对应的干净参考图。本节以每幅输入图像作为新增干扰注入前的参考，叠加频率已知、振幅受控的调幅正弦干扰，并扫描 D0=0.5–8 bin。报告的 PSNR 衡量新增合成干扰的恢复质量；原始周期噪声图则依据频谱峰位置与滤波前后图像变化评价。合成干扰为 $18 \cdot \cos(2\pi(ux/W+vy/H)) \cdot [1+0.5\cos(2\pi \cdot 2x/W)]$，包含载频和两侧调制频率。评价参考是加干扰前的给定图像；$PSNR=10\log10(255^2/MSE)$。半径按最高 PSNR 选取。

表 3 合成噪声搜索的最佳 D0 与 PSNR

图像	搜索最优 D0 (bin)	PSNR (dB)
7.PNG	5.0	51.169
10.jpg	3.0	45.109
11.jpg	3.0	45.977
16.jpg	5.0	53.133

图 9 四幅图各自的受控调幅噪声 D0 搜索曲线：虚线标出本次搜索的最高 PSNR 点



在这次合成信号和搜索网格上，7.PNG 与 16.jpg 的最高点位于 5 bin，10.jpg 与 11.jpg 位于 3 bin。曲线先升后降：过小半径保留调制边带，半径继续增大则抑制邻近图像频率成分。真实四图的陷波参数仍根据各自频谱峰位置和输出图像选择，见表 2；两组 D0 的用途不同。

实验小结

这次实验把径向滤波和陷波的分工表现得很清楚:径向低通或高通按距频谱中心的半径统一改变频率分量;陷波则围绕少数周期噪声峰局部衰减。低通 D0 增大后细节保留更多，理想低通的锐利频域边界会在空间边缘附近带来振铃。高频强调以 0.6 倍低频基线叠加 1.4 倍高通分量，提高本次测试场景的边缘对比。

实现中，频率坐标需要与 fftshift 后的数组索引保持一致;每个噪声峰与其共轭位置成对处理，使传递函数保持共轭对称。高通响应跨越正负灰度，展示图按每幅响应的 1% - 99% 百分位拉伸; CSV 记录未拉伸响应的极值。

PSNR 结果覆盖两组具有已知加噪前参考的实验: cameraman.tif 高斯噪声滤波，以及四张输入图上的受控合成陷波搜索。四幅实际周期噪声图则依据频谱峰位置和滤波前后图像变化评价。

附录 A 频域滤波核心函数：src/frequency_filters.py

```
"""Frequency-domain image filters used in the course experiment."""

from __future__ import annotations

from typing import Iterable

import numpy as np


def frequency_coordinates(shape: tuple[int, int]) -> tuple[np.ndarray, np.ndarray]:
    """Return centered row and column frequency-bin coordinates."""
    height, width = shape
    rows = np.arange(height, dtype=np.float64) - height // 2
    columns = np.arange(width, dtype=np.float64) - width // 2
    return np.meshgrid(rows, columns, indexing="ij")


def manual_dft2(image: np.ndarray) -> np.ndarray:
    """Compute a 2-D DFT directly from the DFT definition using separability."""
    image = np.asarray(image, dtype=np.float64)
    if image.ndim != 2:
        raise ValueError("manual_dft2 expects a two-dimensional grayscale image")
    height, width = image.shape
    row_transform = np.exp(
        -2j * np.pi * np.outer(np.arange(height), np.arange(height)) / height
    )
    column_transform = np.exp(
        -2j * np.pi * np.outer(np.arange(width), np.arange(width)) / width
    )
    return row_transform @ image @ column_transform.T


def radial_filter(
    shape: tuple[int, int], d0: float, kind: str, order: int = 2
) -> np.ndarray:
    """Build an ideal, Butterworth, or Gaussian low/high-pass transfer mask.

    ``d0`` is measured in frequency bins from the centered zero-frequency bin.
    ``kind`` accepts ``ideal_lp``, ``butterworth_lp``, ``gaussian_lp``,
    ``ideal_hp``, ``butterworth_hp``, and ``gaussian_hp``.
    """
    if not np.isfinite(d0) or d0 <= 0:
        raise ValueError("d0 must be a finite positive number")
    if not isinstance(order, int) or order < 1:
        raise ValueError("order must be a positive integer")

    rows, columns = frequency_coordinates(shape)
    distance = np.hypot(rows, columns)
    if kind.endswith("_lp"):
        family = kind[:-3]
        low_pass = True
    elif kind.endswith("_hp"):
        family = kind[:-3]
        low_pass = False
    else:
        raise ValueError("kind must name a low-pass or high-pass filter")

    if family == "ideal":
        low = (distance <= d0).astype(np.float64)
    elif family == "butterworth":
        low = 1.0 / (1.0 + np.power(distance / d0, 2 * order))
    elif family == "gaussian":
        low = np.exp(-(distance**2) / (2.0 * d0**2))
    else:
        raise ValueError("filter family must be ideal, butterworth, or gaussian")

    if low_pass:
        return low
    return 1.0 - low
```

```
def high_frequency_emphasis(
    shape: tuple[int, int], d0: float, a: float = 0.5, b: float = 1.5,
    family: str = "butterworth", order: int = 2,
) -> np.ndarray:
    """Return ``a + b * H_hp`` for high-frequency emphasis enhancement."""
    if not np.isfinite(a) or not np.isfinite(b) or a < 0 or b < 0:
        raise ValueError("a and b must be finite, nonnegative numbers")
    return a + b * radial_filter(shape, d0, family + "_hp", order=order)


def gaussian_notch_reject(
    shape: tuple[int, int], centers: Iterable[tuple[int, int]], d0: float
) -> np.ndarray:
    """Build Gaussian notch rejects for conjugate peak pairs.

    Each center is one representative ``(v, u)`` pair in signed centered-bin
    coordinates: ``v`` is the row offset, ``u`` the column offset. Its
    conjugate ``(-v, -u)`` is included automatically. Supply each pair once.
    """
    if not np.isfinite(d0) or d0 <= 0:
        raise ValueError("d0 must be a finite positive number")
    rows, columns = frequency_coordinates(shape)
    transfer = np.ones(shape, dtype=np.float64)
    seen: set[tuple[int, int]] = set()
    for center in centers:
        if len(center) != 2:
            raise ValueError("each notch center must be a (v, u) pair")
        v, u = int(center[0]), int(center[1])
        canonical = min((v, u), (-v, -u))
        if canonical in seen:
            continue
        seen.add(canonical)
        d_positive = np.hypot(rows - v, columns - u)
        d_negative = np.hypot(rows + v, columns + u)
        transfer *= 1.0 - np.exp(-(d_positive**2) / (2.0 * d0**2))
        if (v, u) != (-v, -u):
            transfer *= 1.0 - np.exp(-(d_negative**2) / (2.0 * d0**2))
    return transfer


def apply_frequency_filter(image: np.ndarray, transfer: np.ndarray) -> np.ndarray:
    """Apply a centered real transfer mask and return a real-valued image."""
    image = np.asarray(image, dtype=np.float64)
    transfer = np.asarray(transfer, dtype=np.float64)
    if image.ndim != 2 or transfer.shape != image.shape:
        raise ValueError("image and transfer mask must be matching 2-D arrays")
    spectrum = np.fft.fftshift(np.fft.fft2(image))
    result = np.fft.ifft2(np.fft.ifftshift(spectrum * transfer))
    return result.real


def dominant_peak_pairs(
    image: np.ndarray, count: int = 12, min_radius: float = 6.0
) -> list[tuple[int, int, float]]:
    """Return strong conjugate spectral peaks as ``(v, u, magnitude)`` rows.

    The center/DC neighborhood is excluded. This is a proposal list for the
    interactive editor; peak choice remains a user decision.
    """
    image = np.asarray(image, dtype=np.float64)
    if image.ndim != 2:
        raise ValueError("dominant_peak_pairs expects a 2-D grayscale image")
    spectrum = np.fft.fftshift(np.fft.fft2(image - image.mean()))
    magnitude = np.abs(spectrum)
    height, width = image.shape
    rows, columns = frequency_coordinates(image.shape)
    radius = np.hypot(rows, columns)

    local_max = np.ones(image.shape, dtype=bool)
```

```
    for row_shift in (-1, 0, 1):
        for column_shift in (-1, 0, 1):
            if row_shift == 0 and column_shift == 0:
                continue
            shifted = np.roll(magnitude, (row_shift, column_shift), axis=(0, 1))
            local_max &= magnitude >= shifted
    local_max &= radius >= min_radius

    candidates = np.argwhere(local_max)
    if candidates.size == 0:
        return []

    # Keep one point from each conjugate pair; sort deterministically by strength.
    result: list[tuple[int, int, float]] = []
    for row_index, column_index in candidates:
        v = int(row_index - height // 2)
        u = int(column_index - width // 2)
        if (v, u) <= (-v, -u):
            continue
        pair_row = (-v + height // 2) % height
        pair_column = (-u + width // 2) % width
        paired_magnitude = magnitude[pair_row, pair_column]
        current_magnitude = magnitude[row_index, column_index]
        score = float(0.5 * (paired_magnitude + current_magnitude))
        result.append((v, u, score))

    result.sort(key=lambda item: (-item[2], item[0], item[1]))
    return result[:count]


def psnr(reference: np.ndarray, estimate: np.ndarray, data_range: float = 255.0) -> float:
    """Peak signal-to-noise ratio for arrays sharing the same pixel scale."""
    reference = np.asarray(reference, dtype=np.float64)
    estimate = np.asarray(estimate, dtype=np.float64)
    if reference.shape != estimate.shape:
        raise ValueError("reference and estimate must have the same shape")
    mse = float(np.mean((reference - estimate) ** 2))
    if mse == 0:
        return float("inf")
    return 10.0 * float(np.log10((data_range**2) / mse))


def to_uint8(image: np.ndarray) -> np.ndarray:
    """Clip a display/export copy to the standard grayscale pixel range."""
    return np.clip(np.rint(image), 0, 255).astype(np.uint8)
```

附录 B 实验运行与结果生成程序：run_experiment.py

```
"""Run the reproducible frequency-domain image-enhancement experiment."""

from __future__ import annotations

import csv
import json
import platform
import sys
import time
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
matplotlib.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
matplotlib.rcParams["axes.unicode_minus"] = False
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image

from src.frequency_filters import (
    apply_frequency_filter,
    dominant_peak_pairs,
    gaussian_notch_reject,
```

```
    high_frequency_emphasis,
    manual_dft2,
    psnr,
    radial_filter,
    to_uint8,
)


ROOT = Path(__file__).resolve().parent
SOURCE_DIR = ROOT / "data" / "source"
RESULTS_DIR = ROOT / "results"
FILTERS = ("ideal", "butterworth", "gaussian")
CUTOFFS = (8.0, 16.0, 32.0, 64.0)
NOISE_SEED = 20261106
NOISE_SIGMA = 20.0

# Coordinates are manual selections from the centered log spectra, in signed
# FFT-bin offsets (row v, column u). Each point selects a conjugate pair.
REAL_NOTCH_CONFIG = {
    "7.PNG": {"centers": [(13, 22), (25, -44)], "d0": 3.0},
    "10.jpg": {
        # First pass: the four isolated carrier/sum/difference peaks. Second
        # pass: the remaining isolated peak at (1, 125); broaden D0 to cover
        # the visible side-lobes around the carrier clusters.
        "centers": [
            (61, -83), (62, 82), (2, 165), (123, -1), (1, 125),
        ],
        "d0": 8.0,
    },
    "11.jpg": {
        # The visible horizontal-stripe fundamental and its in-band harmonics.
        "centers": [(25, 0), (51, 0), (75, 0)],
        "d0": 5.0,
    },
    "16.jpg": {"centers": [(0, 16), (5, 5)], "d0": 3.0},
}
NOTCH_SWEEP = (0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0, 5.0, 6.0, 8.0)
SYNTHETIC_SIDE_BAND = 2


def load_gray(path: Path) -> np.ndarray:
    with Image.open(path) as image:
        return np.asarray(image.convert("L"), dtype=np.float64)


def save_gray(path: Path, image: np.ndarray) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    Image.fromarray(to_uint8(image), mode="L").save(path)


def display_response(image: np.ndarray) -> np.ndarray:
    """Robustly stretch a signed high-pass response for display only."""
    low, high = np.percentile(image, (1.0, 99.0))
    if high <= low:
        return np.zeros_like(image)
    return np.clip((image - low) / (high - low), 0.0, 1.0)


def log_spectrum(image: np.ndarray, shifted: bool = True) -> np.ndarray:
    spectrum = np.fft.fft2(image)
    if shifted:
        spectrum = np.fft.fftshift(spectrum)
    return np.log1p(np.abs(spectrum))


def save_binary_spectrum(results: Path) -> dict[str, float]:
    image = np.zeros((256, 256), dtype=np.float64)
    image[120:136, 126:130] = 1.0  # 16 rows by 4 columns, centered.
    direct = manual_dft2(image)
    fft = np.fft.fft2(image)
    max_error = float(np.max(np.abs(direct - fft)))
```

```
    shifted = np.fft.fftshift(fft)
    restored = np.fft.ifftshift(shifted)
    reconstructed = np.fft.ifft2(restored).real
    reconstruction_error = float(np.max(np.abs(reconstructed - image)))

    figure, axes = plt.subplots(2, 3, figsize=(13, 8), constrained_layout=True)
    axes[0, 0].imshow(image, cmap="gray", vmin=0, vmax=1)
    axes[0, 0].set_title("16 × 4 二值矩形")
    axes[0, 0].axis("off")

    unshifted_mag = np.log1p(np.abs(fft))
    unshifted_phase = np.angle(fft)
    shifted_mag = np.log1p(np.abs(shifted))
    shifted_phase = np.angle(shifted)
    plot_data = [
        (axes[0, 1], unshifted_mag, "未移位幅度谱 log(1+|F|)", "magma"),
        (axes[0, 2], unshifted_phase, "未移位相位谱", "twilight"),
        (axes[1, 0], shifted_mag, "fftshift 后幅度谱", "magma"),
        (axes[1, 1], shifted_phase, "fftshift 后相位谱", "twilight"),
    ]
    for axis, data, title, cmap in plot_data:
        low, high = (-np.pi, np.pi) if "相位" in title else (0, float(np.max(data)))
        im = axis.imshow(data, cmap=cmap, vmin=low, vmax=high)
        axis.set_title(title)
        axis.set_xlabel("列频率索引")
        axis.set_ylabel("行频率索引")
        figure.colorbar(im, ax=axis, fraction=0.046, pad=0.03)
    axes[1, 2].axis("off")
    axes[1, 2].text(
        0.02,
        0.95,
        "说明\n\n"
        "未移位频谱的零频位于左上角。\n"
        "fftshift 将零频移到中心；\n"
        "ifftshift 还原后可直接逆变换。\n\n"
        f"手写 DFT 最大误差：{max_error:.3e}\n"
        f"逆变换重建最大误差：{reconstruction_error:.3e}",
        va="top",
        fontsize=11,
    )
    figure.savefig(results / "binary_fft_spectrum.png", dpi=180)
    plt.close(figure)
    save_gray(results / "binary_rectangle.png", image * 255.0)
    return {
        "manual_dft_max_abs_error": max_error,
        "inverse_reconstruction_max_abs_error": reconstruction_error,
    }


def save_spectrum_comparison(clean: np.ndarray, noisy: np.ndarray, results: Path) -> None:
    figure, axes = plt.subplots(2, 2, figsize=(10, 8), constrained_layout=True)
    axes[0, 0].imshow(clean, cmap="gray", vmin=0, vmax=255)
    axes[0, 0].set_title("原始 256×256 灰度图")
    axes[0, 1].imshow(noisy, cmap="gray", vmin=0, vmax=255)
    axes[0, 1].set_title(f"加入高斯噪声 σ={NOISE_SIGMA:g}")
    for axis, data, title in (
        (axes[1, 0], clean, "原图中心化幅度谱 log(1+|F|)"),
        (axes[1, 1], noisy, "含噪图中心化幅度谱 log(1+|F|)"),
    ):
        spectrum = log_spectrum(data)
        image_plot = axis.imshow(spectrum, cmap="magma")
        axis.set_title(title)
        figure.colorbar(image_plot, ax=axis, fraction=0.046, pad=0.03)
    for axis in axes.ravel():
        axis.axis("off")
    figure.suptitle("高斯噪声加入前后的频谱", fontsize=14)
    figure.savefig(results / "gaussian_noise_spectra.png", dpi=180)
    plt.close(figure)
```

```
def save_ideal_ringing_demo(scene: np.ndarray, results: Path) -> dict[str, int]:
    # Select a strong edge in the central scene region, then compare a crop and
    # its cross-section under the same cutoff radius.
    region = np.abs(np.diff(scene[80:175, 30:160], axis=1))
    relative_row, relative_column = np.unravel_index(np.argmax(region), region.shape)
    row = int(relative_row + 80)
    edge_column = int(relative_column + 31)
    d0 = 16.0
    ideal = apply_frequency_filter(scene, radial_filter(scene.shape, d0, "ideal_lp"))
    gaussian = apply_frequency_filter(scene, radial_filter(scene.shape, d0, "gaussian_lp"))

    row0, row1 = max(0, row - 10), min(scene.shape[0], row + 11)
    col0, col1 = max(0, edge_column - 22), min(scene.shape[1], edge_column + 23)
    crop = (slice(row0, row1), slice(col0, col1))
    x_values = np.arange(col0, col1)
    figure, axes = plt.subplots(2, 2, figsize=(11, 8), constrained_layout=True)
    for axis, image, title in (
        (axes[0, 0], scene, "原始场景（局部边缘位置）"),
        (axes[0, 1], ideal, f"理想低通 D0={d0:g}（裁剪）"),
        (axes[1, 0], gaussian, f"高斯低通 D0={d0:g}（裁剪）"),
    ):
        axis.imshow(image[crop], cmap="gray", vmin=0, vmax=255)
        axis.axvline(edge_column - col0, color="#d62728", linewidth=1, linestyle="--")
        axis.set_title(title)
        axis.axis("off")
    axes[1, 1].plot(x_values, scene[row, col0:col1], label="原图", color="#444444")
    axes[1, 1].plot(x_values, ideal[row, col0:col1], label="理想低通", color="#c44e52")
    axes[1, 1].plot(x_values, gaussian[row, col0:col1], label="高斯低通", color="#1c6e8c")
    axes[1, 1].axvline(edge_column, color="#777777", linestyle="--", linewidth=1)
    axes[1, 1].set_title(f"第 {row} 行边缘剖面（红线为强边缘）")
    axes[1, 1].set_xlabel("列坐标")
    axes[1, 1].set_ylabel("灰度")
    axes[1, 1].grid(alpha=0.25)
    axes[1, 1].legend(frameon=False)
    figure.savefig(results / "ideal_lowpass_ringing.png", dpi=180)
    plt.close(figure)
    return {"edge_row": row, "edge_column": edge_column, "D0_bins": int(d0)}


def run_gaussian_noise(results: Path) -> tuple[list[dict[str, object]], np.ndarray, np.ndarray]:
    clean = load_gray(SOURCE_DIR / "cameraman.tif")
    if clean.shape != (256, 256):
        raise ValueError(f"cameraman.tif must be 256x256; got {clean.shape}")
    rng = np.random.default_rng(NOISE_SEED)
    noisy = np.clip(clean + rng.normal(0.0, NOISE_SIGMA, clean.shape), 0.0, 255.0)
    save_gray(results / "cameraman_gaussian_noisy.png", noisy)
    save_spectrum_comparison(clean, noisy, results)
    ringing_location = save_ideal_ringing_demo(clean, results)

    rows: list[dict[str, object]] = []
    filtered_images: dict[tuple[str, float], np.ndarray] = {}
    for family in FILTERS:
        for cutoff in CUTOFFS:
            mask = radial_filter(clean.shape, cutoff, family + "_lp", order=2)
            start = time.perf_counter()
            filtered = apply_frequency_filter(noisy, mask)
            elapsed = time.perf_counter() - start
            score = psnr(clean, filtered)
            filtered_images[(family, cutoff)] = filtered
            save_gray(results / "lowpass" / f"{family}_D0_{cutoff:g}.png", filtered)
            rows.append(
                {
                    "filter": family,
                    "D0_bins": cutoff,
                    "order": 2 if family == "butterworth" else "n/a",
                    "PSNR_dB": score,
                    "MSE": float(np.mean((clean - filtered) ** 2)),
                    "runtime_seconds": elapsed,
                    "reference": "cameraman.tif before added Gaussian noise",
                }
            )
```

```
    with (results / "gaussian_lowpass_metrics.csv").open("w", newline="", encoding="utf-8-sig") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)

    noisy_psnr = psnr(clean, noisy)
    figure, axes = plt.subplots(3, 4, figsize=(14, 10), constrained_layout=True)
    for row_index, family in enumerate(FILTERS):
        for column_index, cutoff in enumerate(CUTOFFS):
            output = filtered_images[(family, cutoff)]
            score = psnr(clean, output)
            axis = axes[row_index, column_index]
            axis.imshow(output, cmap="gray", vmin=0, vmax=255)
            axis.set_title(f"{family.title()} LP · D0={cutoff:g}\nPSNR={score:.2f} dB")
            axis.axis("off")
    figure.suptitle(
        f"高斯噪声低通结果（σ={NOISE_SIGMA:g}，seed={NOISE_SEED}；含噪 PSNR={noisy_psnr:.2f} dB）",
        fontsize=14,
    )
    figure.savefig(results / "gaussian_lowpass_grid.png", dpi=180)
    plt.close(figure)

    summary = {
        "reference": "cameraman.tif before added Gaussian noise",
        "noise_model": "independent zero-mean Gaussian, then clipped to [0,255]",
        "sigma": NOISE_SIGMA,
        "seed": NOISE_SEED,
        "noisy_psnr_db": noisy_psnr,
        "ideal_lowpass_ringing_edge": ringing_location,
    }
    (results / "gaussian_noise_metadata.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    return rows, clean, noisy


def run_highpass(results: Path, scene: np.ndarray) -> list[dict[str, object]]:
    rows: list[dict[str, object]] = []
    filtered_images: dict[tuple[str, float], np.ndarray] = {}
    for family in FILTERS:
        for cutoff in CUTOFFS:
            mask = radial_filter(scene.shape, cutoff, family + "_hp", order=2)
            response = apply_frequency_filter(scene, mask)
            filtered_images[(family, cutoff)] = response
            save_gray(results / "highpass" / f"{family}_D0_{cutoff:g}_stretched.png", display_response(response)
* 255.0)
            rows.append(
                {
                    "filter": family,
                    "D0_bins": cutoff,
                    "order": 2 if family == "butterworth" else "n/a",
                    "response_min": float(response.min()),
                    "response_max": float(response.max()),
                    "display_mapping": "1st-99th percentile linear stretch for PNG; CSV stores only raw response
min/max",
                }
            )

    with (results / "highpass_parameters.csv").open("w", newline="", encoding="utf-8-sig") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)

    figure, axes = plt.subplots(3, 4, figsize=(14, 10), constrained_layout=True)
    for row_index, family in enumerate(FILTERS):
        for column_index, cutoff in enumerate(CUTOFFS):
            axis = axes[row_index, column_index]
            axis.imshow(display_response(filtered_images[(family, cutoff)]), cmap="gray", vmin=0, vmax=1)
            axis.set_title(f"{family.title()} HP · D0={cutoff:g}")
            axis.axis("off")
```

```
    figure.suptitle("自然场景高通响应（每幅响应单独做 1%–99% 显示拉伸）", fontsize=14)
    figure.savefig(results / "highpass_grid.png", dpi=180)
    plt.close(figure)

    # HFE retains a low-frequency baseline while strengthening high-frequency detail.
    hfe_mask = high_frequency_emphasis(
        scene.shape, d0=24.0, a=0.6, b=1.4, family="butterworth", order=2
    )
    enhanced = apply_frequency_filter(scene, hfe_mask)
    save_gray(results / "high_frequency_emphasis.png", enhanced)
    figure, axes = plt.subplots(1, 3, figsize=(13, 4.5), constrained_layout=True)
    axes[0].imshow(scene, cmap="gray", vmin=0, vmax=255)
    axes[0].set_title("原始 cameraman.tif")
    axes[1].imshow(apply_frequency_filter(scene, radial_filter(scene.shape, 24.0, "butterworth_hp", 2)),
cmap="gray")
    axes[1].set_title("巴特沃斯高通响应（自动显示范围）")
    axes[2].imshow(enhanced, cmap="gray", vmin=0, vmax=255)
    axes[2].set_title("高频强调：a=0.6, b=1.4, D0=24, n=2")
    for axis in axes:
        axis.axis("off")
    figure.savefig(results / "high_frequency_emphasis_comparison.png", dpi=180)
    plt.close(figure)
    return rows


def synthetic_modulated_noise(
    shape: tuple[int, int], center: tuple[int, int], side_band: int, amplitude: float = 18.0
) -> np.ndarray:
    """Known sinusoidal interference with two nearby modulation sidebands."""
    height, width = shape
    v, u = center
    rows, columns = np.mgrid[0:height, 0:width]
    carrier_phase = 2.0 * np.pi * (u * columns / width + v * rows / height)
    modulation = np.cos(2.0 * np.pi * side_band * columns / width)
    return amplitude * np.cos(carrier_phase) * (1.0 + 0.5 * modulation)


def plot_peaks(
    image: np.ndarray,
    selected_centers: list[tuple[int, int]],
    title: str,
    path: Path,
) -> None:
    spectrum = log_spectrum(image)
    height, width = image.shape
    figure, axis = plt.subplots(figsize=(8.5, 6.0), constrained_layout=True)
    extent = (-(width // 2) - 0.5, (width - 1 - width // 2) + 0.5,
              -(height // 2) - 0.5, (height - 1 - height // 2) + 0.5)
    image_plot = axis.imshow(spectrum, origin="lower", extent=extent, cmap="magma")
    for index, (v, u) in enumerate(selected_centers, start=1):
        axis.scatter([u, -u], [v, -v], marker="o", s=46, facecolors="none", edgecolors="cyan", linewidths=1.3)
        axis.annotate(str(index), (u, v), xytext=(4, 4), textcoords="offset points", color="white", fontsize=9)
    axis.set_title(title)
    axis.set_xlabel("水平频率 u（bin）")
    axis.set_ylabel("垂直频率 v（bin）")
    figure.colorbar(image_plot, ax=axis, label="log(1 + |F|)")
    figure.savefig(path, dpi=180)
    plt.close(figure)


def run_notch_challenge(results: Path) -> tuple[list[dict[str, object]], list[dict[str, object]]]:
    challenge_files = ("7.PNG", "10.jpg", "11.jpg", "16.jpg")
    peak_rows: list[dict[str, object]] = []
    synthetic_rows: list[dict[str, object]] = []
    actual_rows: list[dict[str, object]] = []
    notch_outputs: list[tuple[str, np.ndarray, np.ndarray, np.ndarray, float]] = []

    for image_index, filename in enumerate(challenge_files):
        original = load_gray(SOURCE_DIR / filename)
        height, width = original.shape
        peaks = dominant_peak_pairs(original, count=16, min_radius=6.0)
```

```
for rank, (v, u, magnitude) in enumerate(peaks, start=1):
    peak_rows.append(
        {
            "image": filename,
            "height": height,
            "width": width,
            "rank": rank,
            "row_frequency_v": v,
            "column_frequency_u": u,
            "fft_magnitude": magnitude,
            "selected_for_notch": (v, u) in REAL_NOTCH_CONFIG[filename]["centers"],
        }
    )
plot_peaks(
    original,
    REAL_NOTCH_CONFIG[filename]["centers"],
    f"{filename}：人工选取的频谱峰（青圈为陷波中心）",
    results / f"notch_spectrum_{Path(filename).stem}.png",
)

# PSNR sweep uses an exact, known pre-injection reference. It measures
# radius choice for this controlled sinusoidal corruption only.
center = (max(9, int(round(height * 0.13))), max(11, int(round(width * 0.11))))
if image_index % 2:
    center = (center[0], -center[1])
controlled_noise = synthetic_modulated_noise(
    original.shape, center, SYNTHETIC_SIDE_BAND, amplitude=18.0
)
controlled_noisy = original + controlled_noise
for d0 in NOTCH_SWEEP:
    transfer = gaussian_notch_reject(original.shape, [center], d0)
    restored = apply_frequency_filter(controlled_noisy, transfer)
    synthetic_rows.append(
        {
            "image": filename,
            "D0_bins": d0,
            "carrier_v": center[0],
            "carrier_u": center[1],
            "sideband_bins": SYNTHETIC_SIDE_BAND,
            "amplitude": 18.0,
            "PSNR_dB": psnr(original, restored),
            "MSE": float(np.mean((original - restored) ** 2)),
            "reference": "provided input image before synthetic noise injection",
        }
    )

config = REAL_NOTCH_CONFIG[filename]
real_transfer = gaussian_notch_reject(original.shape, config["centers"], config["d0"])
filtered = apply_frequency_filter(original, real_transfer)
save_gray(results / "notch_filtered" / f"{Path(filename).stem}_notch.png", filtered)
residual_spectrum = log_spectrum(filtered)
figure, axes = plt.subplots(1, 3, figsize=(14, 4.5), constrained_layout=True)
axes[0].imshow(original, cmap="gray", vmin=0, vmax=255)
axes[0].set_title(f"输入：{filename}")
axes[1].imshow(filtered, cmap="gray", vmin=0, vmax=255)
axes[1].set_title(f"陷波结果：D0={config['d0']:g} bin")
axes[2].imshow(residual_spectrum, cmap="magma")
axes[2].set_title("滤波后 log(1+|F|)")
for axis in axes:
    axis.axis("off")
figure.savefig(results / f"notch_result_{Path(filename).stem}.png", dpi=180)
plt.close(figure)

actual_rows.append(
    {
        "image": filename,
        "height": height,
        "width": width,
        "selected_peak_pairs_v_u": json.dumps(config["centers"]),
        "D0_bins": config["d0"],
        "filter": "Gaussian notch reject",
```

```
            "assessment": "visual/spectral only; no clean reference supplied",
            "source_min": float(original.min()),
            "source_max": float(original.max()),
            "filtered_min_before_clip": float(filtered.min()),
            "filtered_max_before_clip": float(filtered.max()),
        }
    )
    notch_outputs.append((filename, original, filtered, real_transfer, config["d0"]))

with (results / "notch_peak_candidates.csv").open("w", newline="", encoding="utf-8-sig") as handle:
    writer = csv.DictWriter(handle, fieldnames=list(peak_rows[0]))
    writer.writeheader()
    writer.writerows(peak_rows)
with (results / "notch_psnr_sweep.csv").open("w", newline="", encoding="utf-8-sig") as handle:
    writer = csv.DictWriter(handle, fieldnames=list(synthetic_rows[0]))
    writer.writeheader()
    writer.writerows(synthetic_rows)
with (results / "notch_real_image_parameters.csv").open("w", newline="", encoding="utf-8-sig") as handle:
    writer = csv.DictWriter(handle, fieldnames=list(actual_rows[0]))
    writer.writeheader()
    writer.writerows(actual_rows)

best_rows = []
for filename in challenge_files:
    subset = [row for row in synthetic_rows if row["image"] == filename]
    best = max(subset, key=lambda row: float(row["PSNR_dB"]))
    for row in subset:
        row["is_best"] = row is best
    best_rows.append(best)

figure, axes = plt.subplots(2, 2, figsize=(11, 8), constrained_layout=True)
for axis, filename in zip(axes.ravel(), challenge_files):
    subset = [row for row in synthetic_rows if row["image"] == filename]
    x = [float(row["D0_bins"]) for row in subset]
    y = [float(row["PSNR_dB"]) for row in subset]
    best = max(subset, key=lambda row: float(row["PSNR_dB"]))
    axis.plot(x, y, marker="o", color="#1c6e8c")
    axis.axvline(float(best["D0_bins"]), color="#c44e52", linestyle="--", linewidth=1)
    axis.set_title(f"{filename} · best D0={float(best['D0_bins']):g} bin")
    axis.set_xlabel("D0（频率 bin）")
    axis.set_ylabel("PSNR（dB）")
    axis.grid(alpha=0.25)
figure.suptitle("已知调幅周期噪声上的陷波半径 PSNR 搜索", fontsize=14)
figure.savefig(results / "notch_psnr_sweep.png", dpi=180)
plt.close(figure)

best_summary = [
    {
        "image": row["image"],
        "best_D0_bins": row["D0_bins"],
        "best_PSNR_dB": row["PSNR_dB"],
        "reference": row["reference"],
    }
    for row in best_rows
]
(results / "notch_psnr_best.json").write_text(
    json.dumps(best_summary, ensure_ascii=False, indent=2), encoding="utf-8"
)

# Display all four real-image outputs together, preserving each source's
# aspect ratio and dimensions.
figure, axes = plt.subplots(4, 3, figsize=(13, 13), constrained_layout=True)
for row_index, (filename, original, filtered, transfer, d0) in enumerate(notch_outputs):
    axes[row_index, 0].imshow(original, cmap="gray", vmin=0, vmax=255)
    axes[row_index, 0].set_title(f"{filename} 原图")
    axes[row_index, 1].imshow(to_uint8(filtered), cmap="gray", vmin=0, vmax=255)
    axes[row_index, 1].set_title(f"Gaussian notch，D0={d0:g}")
    axes[row_index, 2].imshow(1.0 - transfer, cmap="magma", vmin=0, vmax=1)
    axes[row_index, 2].set_title("陷波衰减量 1−H")
    for axis in axes[row_index]:
        axis.axis("off")
```

```
    figure.suptitle("四张周期噪声图的实际陷波结果", fontsize=14)
    figure.savefig(results / "notch_all_four.png", dpi=180)
    plt.close(figure)

    return actual_rows, synthetic_rows


def main() -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    (RESULTS_DIR / "lowpass").mkdir(exist_ok=True)
    (RESULTS_DIR / "highpass").mkdir(exist_ok=True)
    (RESULTS_DIR / "notch_filtered").mkdir(exist_ok=True)

    fft_metrics = save_binary_spectrum(RESULTS_DIR)
    lowpass_rows, clean, noisy = run_gaussian_noise(RESULTS_DIR)
    highpass_rows = run_highpass(RESULTS_DIR, clean)
    notch_rows, notch_sweep_rows = run_notch_challenge(RESULTS_DIR)

    metadata = {
        "python": sys.version,
        "platform": platform.platform(),
        "numpy": np.__version__,
        "pillow": Image.__version__ if hasattr(Image, "__version__") else __import__("PIL").__version__,
        "matplotlib": matplotlib.__version__,
        "random_seed": NOISE_SEED,
        "gaussian_sigma": NOISE_SIGMA,
        "cutoff_bins": list(CUTOFFS),
        "butterworth_order": 2,
        "binary_fft": fft_metrics,
        "gaussian_noisy_psnr_db": psnr(clean, noisy),
        "gaussian_lowpass_rows": len(lowpass_rows),
        "highpass_rows": len(highpass_rows),
        "notch_real_images": len(notch_rows),
        "notch_synthetic_psnr_rows": len(notch_sweep_rows),
        "real_periodic_images_psnr": "not computed: clean reference images were not supplied",
        "synthetic_notch_psnr_reference": "each supplied image before the controlled modulated sinusoidal noise
was added",
        "hfe_parameters": {"family": "butterworth", "order": 2, "D0_bins": 24.0, "a": 0.6, "b": 1.4},
        "actual_notch_parameters": REAL_NOTCH_CONFIG,
    }
    (RESULTS_DIR / "run_metadata.json").write_text(
        json.dumps(metadata, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(json.dumps(metadata, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

附录 C 交互式陷波程序：src/notch_editor.py

```
"""Interactive Gaussian notch editor for the four periodic-noise images."""

from __future__ import annotations

import argparse
import json
from pathlib import Path

import matplotlib

if matplotlib.get_backend().lower() in {"agg", "pdf", "svg", "ps", "template"}:
    try:
        matplotlib.use("TkAgg")
    except ImportError as exc:  # pragma: no cover - depends on workstation GUI
        raise RuntimeError("An interactive Matplotlib backend is required (TkAgg recommended)") from exc

matplotlib.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
matplotlib.rcParams["axes.unicode_minus"] = False
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.widgets import Button, Slider
```

```
from PIL import Image

from .frequency_filters import (
    apply_frequency_filter,
    dominant_peak_pairs,
    gaussian_notch_reject,
    to_uint8,
)


def load_gray(path: Path) -> np.ndarray:
    with Image.open(path) as image:
        return np.asarray(image.convert("L"), dtype=np.float64)


def image_extent(shape: tuple[int, int]) -> tuple[float, float, float, float]:
    height, width = shape
    left = -(width // 2) - 0.5
    right = (width - 1 - width // 2) + 0.5
    bottom = -(height // 2) - 0.5
    top = (height - 1 - height // 2) + 0.5
    return left, right, bottom, top


def show_editor(image_path: Path, output_path: Path, initial_d0: float = 3.0) -> None:
    original = load_gray(image_path)
    height, width = original.shape
    spectrum = np.log1p(np.abs(np.fft.fftshift(np.fft.fft2(original - original.mean()))))
    suggestions = dominant_peak_pairs(original, count=10, min_radius=6.0)
    centers: list[tuple[int, int]] = []

    figure, axes = plt.subplots(1, 3, figsize=(15, 6))
    figure.subplots_adjust(bottom=0.23, wspace=0.18, left=0.05, right=0.97)
    axes[0].imshow(original, cmap="gray", vmin=0, vmax=255)
    axes[0].set_title(f"输入：{image_path.name}")
    axes[0].axis("off")

    extent = image_extent(original.shape)
    spectrum_artist = axes[1].imshow(
        spectrum, origin="lower", extent=extent, cmap="magma"
    )
    axes[1].set_title("点击频谱峰添加陷波中心；右键删除最近一组")
    axes[1].set_xlabel("水平频率 u（bin）")
    axes[1].set_ylabel("垂直频率 v（bin）")
    figure.colorbar(spectrum_artist, ax=axes[1], fraction=0.046, pad=0.03)

    initial_output = original.copy()
    result_artist = axes[2].imshow(initial_output, cmap="gray", vmin=0, vmax=255)
    axes[2].set_title("滤波结果")
    axes[2].axis("off")
    marks = []
    instructions = (
        "建议峰值（按强度）：\n"
        + ", ".join(f"({v},{u})" for v, u, _ in suggestions[:6])
        + "\n左键在频谱中选峰；单侧选择会自动加入共轭点。"
    )
    figure.text(0.34, 0.04, instructions, ha="center", va="bottom", fontsize=9)

    slider_axis = figure.add_axes([0.13, 0.15, 0.53, 0.035])
    radius_slider = Slider(slider_axis, "D0 (bin)", 0.5, 20.0, valinit=initial_d0, valstep=0.5)
    save_axis = figure.add_axes([0.73, 0.12, 0.1, 0.07])
    save_button = Button(save_axis, "保存结果")
    clear_axis = figure.add_axes([0.85, 0.12, 0.1, 0.07])
    clear_button = Button(clear_axis, "清空陷波")
    status = figure.text(0.72, 0.21, "陷波中心：无", ha="center", fontsize=9)

    def draw_centers() -> None:
        nonlocal marks
        for mark in marks:
            mark.remove()
        marks = []
```

```
        for v, u in centers:
            marks.append(
                axes[1].scatter(
                    [u, -u], [v, -v], marker="o", s=70,
                    facecolors="none", edgecolors="#41f2d5", linewidths=1.7,
                )
            )
        status.set_text("陷波中心：(v,u) " + (", ".join(map(str, centers)) if centers else "无"))

    def recompute(_value: object = None) -> None:
        d0 = float(radius_slider.val)
        if centers:
            transfer = gaussian_notch_reject(original.shape, centers, d0)
            filtered = apply_frequency_filter(original, transfer)
        else:
            filtered = original
        result_artist.set_data(to_uint8(filtered))
        axes[2].set_title(f"Gaussian notch • D0={d0:g} bin • {len(centers)} 组频率")
        figure.canvas.draw_idle()

    def on_click(event: object) -> None:
        if event.inaxes is not axes[1] or event.xdata is None or event.ydata is None:
            return
        u, v = int(round(event.xdata)), int(round(event.ydata))
        if np.hypot(v, u) < 5:
            status.set_text("中心低频区已忽略；请点选远离原点的亮峰。")
            figure.canvas.draw_idle()
            return
        canonical = min((v, u), (-v, -u))
        if event.button == 1:
            if not any(min(center, (-center[0], -center[1])) == canonical for center in centers):
                centers.append((v, u))
                draw_centers()
                recompute()
        elif event.button == 3 and centers:
            distances = [min(np.hypot(v - cv, u - cu), np.hypot(v + cv, u + cu)) for cv, cu in centers]
            closest = int(np.argmin(distances))
            if distances[closest] <= 8.0:
                centers.pop(closest)
                draw_centers()
                recompute()

    def save_result(_event: object) -> None:
        d0 = float(radius_slider.val)
        transfer = gaussian_notch_reject(original.shape, centers, d0)
        filtered = apply_frequency_filter(original, transfer)
        output_path.parent.mkdir(parents=True, exist_ok=True)
        Image.fromarray(to_uint8(filtered), mode="L").save(output_path)
        parameter_path = output_path.with_suffix(".json")
        parameter_path.write_text(
            json.dumps(
                {
                    "input": str(image_path),
                    "output": str(output_path),
                    "image_shape": list(original.shape),
                    "centers_v_u": centers,
                    "D0_bins": d0,
                    "filter": "Gaussian notch reject",
                },
                ensure_ascii=False,
                indent=2,
            ),
            encoding="utf-8",
        )
        status.set_text(f"已保存：{output_path.name}")
        figure.canvas.draw_idle()

    def clear_centers(_event: object) -> None:
        centers.clear()
        draw_centers()
        recompute()
```

```
    radius_slider.on_changed(recompute)
    figure.canvas.mpl_connect("button_press_event", on_click)
    save_button.on_clicked(save_result)
    clear_button.on_clicked(clear_centers)
    plt.show()


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--image", required=True, type=Path, help="input grayscale image")
    parser.add_argument("--output", type=Path, default=Path("notch_filtered.png"))
    parser.add_argument("--d0", type=float, default=3.0, help="initial notch radius in FFT bins")
    args = parser.parse_args()
    show_editor(args.image, args.output, initial_d0=args.d0)


if __name__ == "__main__":
    main()
```